When Randomness Isn't Random: Practical Fault Attack on Post-Quantum Lattice Standards

Hariprasad Kelassery Valsaraj*‡, Prasanna Ravi†¶, Shivam Bhasin‡§, Hongjun Wu*
*School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore
†College of Computing and Data Science, Nanyang Technological University, Singapore
‡Temasek Lab, Nanyang Technological University, Singapore
§National Integrated Centre For Evaluation, Nanyang Technological University, Singapore.
¶PQStation Pte. Ltd, Singapore
Emails: haripras003@e.ntu.edu.sg, prasanna.ravi@ntu.edu.sg, sbhasin@ntu.edu.sg, wuhj@ntu.edu.sg

***Abstract*—Post-quantum cryptographic schemes like ML-KEM and ML-DSA have been standardized to secure digital communication against quantum threats. While their theoretical foundations are robust, we identify a critical implementation-level vulnerability in both: a single point of failure centered on the random seed pointer used in polynomial sampling. By corrupting this pointer, an attacker can deterministically compromise the entire scheme, bypassing standard countermeasures. We present the first practical fault-injection attacks exploiting this weakness and validate them on an STM32H7 microcontroller using laser fault injection. Our results demonstrate full key and message recovery for ML-KEM and signature forgery for ML-DSA, with success rates up to 100%. We further verify the presence of this vulnerable implementation style in widely used public libraries, including PQM4, LibOQS, PQClean, and WolfSSL, and propose effective countermeasures to mitigate this overlooked yet severe threat.**

***Index Terms*—Post quantum cryptography, ML-KEM, ML-DSA, laser fault injection**

1. Introduction

The impending quantum threat is becoming increasingly tangible with rapid advancements in quantum computing technologies [9]. As we approach the so-called Years to Quantum (Y2Q) moment—the point at which large-scale quantum computers will be capable of breaking widely used public-key cryptographic schemes such as RSA and ECC within a single day[33]—the urgency for quantum-resistant alternatives is growing [9]. Fortunately, the cryptographic community has been proactive in addressing this risk. Since 2017, the National Institute of Standards and Technology (NIST) has led a rigorous and transparent standardization effort to evaluate and select post-quantum cryptographic algorithms [4]. This process, which involved global contributions from academia and industry, has culminated in the selection of five primary standards: ML-KEM[19] and HQC[17] for key encapsulation, and ML-DSA[18], SLH-DSA[20], and FN-DSA[8] for digital signatures.

With the finalization of NIST's PQC standards, real-world deployment of these algorithms has already begun across a wide range of platforms and applications. ML-KEM and ML-DSA, in particular, are seeing early adoption in mainstream environments including Google Chrome [10], Zoom [35], Apple iMessage, and Cloudflare's web infrastructure [5]. These algorithms are also making their way into embedded systems, with security libraries such as WolfSSL[31] actively integrating post-quantum support into their offerings for resource-constrained and high-assurance environments. Given the ever expanding list of attack vectors particularly on embedded devices, it becomes critically important to study the security of practical implementations of ML-KEM and ML-DSA on resource constrained platforms.

During the NIST Post-Quantum Cryptography (PQC) standardization process, significant research efforts were dedicated to evaluating the susceptibility of proposed algorithms—particularly ML-KEM and ML-DSA—to Side-Channel Attacks (SCA) and Fault-Injection Attacks (FIA). Among these, fault attacks have drawn considerable attention, with numerous studies demonstrating practical key and message recovery through various fault models, ranging from single to multiple fault injections across multiple executions [25], [27], [3]. These attacks target diverse operations within the cryptographic implementations. A central insight from these works [23] is that lattice-based schemes inherently offer a wide surface area for fault exploitation. Given the breadth of possible attack vectors, our investigation focuses on identifying the most straightforward and intuitive target for a fault attacker. Our analysis reveals that the seed used to derive all key material—both public and private—presents an especially attractive and vulnerable entry point.

The core vulnerability arises from the mechanism through which ML-KEM and ML-DSA generate the randomness necessary for polynomial sampling, a critical component of their underlying Learning With Errors (LWE)-based constructions. Both schemes utilize Keccak-based Extendable Output Functions (XOFs) to expand a short, truly random seed into the required pseudorandom values. This introduces a critical dependency: the seed serves as a *single*

TABLE 1: Fault-injection attacks on Lattice Schemes and proposed countermeasures (data sourced from [23]).

Attack	Targeted Procedure	Proposed countermeasure	Bypassed
Attacks on ML-KEM			
Nonce_Fault[26]	KeyGen, Encaps	Verify_Nonce_Fault	✓
NTT_Twiddle_Fault[27]	KeyGen, Encaps	NTT_Twiddle_Check	✓
Skip_CT_Compare[34]	Decaps	Protect_CT_Compare	NA
Ineffective_FIA[22]	Decaps	Shuffled_Decode	NA
Fault_Correction[6], [11]	Decaps	Message_Poly_Sanity_Check	NA
Attacks on ML-DSA			
Nonce_Fault[26]	KeyGen	Verify_Nonce_Fault	✓
NTT_Twiddle_Fault[27]	Sign, Verify	NTT_Twiddle_Check	✓
Randomize_Secret_Key[2], [12]	KeyGen	Verify_After_Sign	✓
Generic_DFA[3]	Sign	Verify_After_Sign	✓
Loop_Abort_Fault[7]	Sign	Verify_Loop_Abort	✓
Skip_Addition[2], [24]	Sign	Verify_Add	✓
Skip_C_Compare[34]	Verify	Dynamic_Loop_Counter	NA

point of failure. If an attacker can corrupt the pointer to this seed, the cryptographic integrity of the scheme can be fatally undermined. Our experimental results demonstrate that such seed pointer corruption enables high-probability key and message recovery attacks on ML-KEM, as well as signature forgery attacks on ML-DSA, with observed success rates ranging from 90% to 100%.

Notably, prior research has not explored attacks specifically targeting this seed value, and specifically the pointer to the seed. In this work, we present novel fault attacks on ML-KEM and ML-DSA that target the seed pointer and systematically analyze the classes of attacks that arise from its corruption, emphasizing its role as a critical single point of failure in both schemes.

1.1. Contribution

The key contributions of this work are as follow:

1) We present the first practical fault attacks targeting the pointer to the *critical seed* pointer used in both ML-KEM and ML-DSA. Our research specifically identifies code locations where corrupting this pointer creates a single point of failure, leading to effective key recovery and forgery attacks.
2) We practically validate these attacks on an STM32H7 microcontroller using laser fault injection, demonstrating their effectiveness against the publicly available PQM4 library. Our experiments show that these vulnerabilities can lead to full secret key and message recovery for ML-KEM, and signature forgery against ML-DSA. We also confirm this vulnerability across other widely used public libraries such as PQM4 [13] and LibOQS [21].
3) Our demonstrated fault model bypasses most known countermeasures for lattice-based cryptographic schemes (Table 1), highlighting a significant and previously underexplored vulnerability in current post-quantum cryptographic defenses.
4) We also discuss and propose effective countermeasures to strengthen the manipulation of seed pointer, in order to mitigate our proposed fault attacks.

2. Background

This section provides the general background on ML-KEM and ML-DSA. We also discuss notable related works related to fault attacks on their embedded implementations.

Notations

For a prime number q, $\mathbb{Z}_q$ denotes the field of integers modulo q. For a positive integer n, the polynomial ring $R_q = \mathbb{Z}_q[x]/\left(x^n + 1\right)$ consists of all polynomials in $\mathbb{Z}_q[x]$ of degree less than n, with addition and multiplication defined modulo $x^n + 1$. Let η be an integer that is significantly smaller than $q/2$. We denote by S_η the set of small polynomials of size η, where the coefficients lie within the range $[-\eta, \eta]$.

2.1. The Learning With Errors (LWE) Problem

Lattice-based key encapsulation mechanisms (KEMs) and digital signature (DS) schemes derive their security from the computational hardness of the *Module Learning With Errors (MLWE)[28]* problem in module lattices. Formally, an MLWE instance is characterized by the pair (A, t), where A is a random $k \times l$ matrix of polynomials over R_q, and $t = A \cdot s + e$. Here, s and e are matrices composed of small polynomials drawn from S_η. The objective of the MLWE problem is to recover s given (A, t). As the small error e prevents an efficient recovery of s, MLWE instances (A, t) can be used as public keys in both KEMs and digital signature schemes.

2.1.1. ML-KEM. ML-KEM refers to the NIST-standardized variant of Kyber, a Chosen Ciphertext Attack (CCA) secure key encapsulation mechanism based on the Module Learning with Errors (MLWE) problem. A

$k \times k$ matrix of polynomials in the ring R_q, where $q = 3329$ and $n = 256$, is used by an LWE instance in ML-KEM. By changing k, different security levels are supported. The ML-KEM-512, ML-KEM-768, and ML-KEM-1024 are specifically for $k = 2$, $k = 3$, and $k = 4$, respectively.

At its core, ML-KEM uses a Chosen-Plaintext Attack (CPA) secure public-key encryption (PKE) scheme. For a simplified version of the PKE key generation we can refer to Algorithm 1.

Algorithm 1 ML-KEM PKE key generation (Simplified)

Input: $d \in \mathcal{B}^{32}$
Output: Secret key $sk \in \mathcal{B}^{384 \cdot k}$
Output: Public key $pk \in \mathcal{B}^{384 \cdot k + 32}$
1: $(\rho, \sigma) \leftarrow G(d)$
2: $N \leftarrow 0$
3: $\hat{A} \leftarrow \text{Expand}(\rho)$ ▷ Generate matrix $\hat{A} \in R_q^{k \times k}$
4: **for** $i = 0$ to $k - 1$ **do** ▷ Sample $s \in R_q^k$ from B_{η_1}
5: $s[i] \leftarrow \text{CBD}_{\eta_1}(\text{PRF}(\sigma, N))$
6: $N \leftarrow N + 1$
7: **end for**
8: **for** $i = 0$ to $k - 1$ **do** ▷ Sample $e \in R_q^k$ from B_{η_1}
9: $e[i] \leftarrow \text{CBD}_{\eta_1}(\text{PRF}(\sigma, N))$
10: $N \leftarrow N + 1$
11: **end for**
12: $\hat{t} \leftarrow \hat{A} \circ \text{NTT}(s) + \text{NTT}(e)$
13: $pk \leftarrow \text{Encode}(\hat{t} \parallel \rho)$
14: $sk \leftarrow \text{Encode}(\hat{s})$
15: **return** (pk, sk)

Since sampling several polynomial coefficients in an LWE instance construction requires a significant amount of randomness, ML-KEM uses optimizations that start with a tiny, truly random seed and grow it using pseudo-random functions (PRF). Specifically, the 32-byte random seed is accepted by the ML-KEM public key encryption (PKE) module and processed by the hash function G (SHA3-512) to produce the public seed ρ and the private seed σ. An extensible output function (XOF), in this case SHAKE-128, is then used to produce the coefficient matrix A from ρ. In the meantime, σ is passed to a PRF (SHAKE-256) to get secret and error vectors, and its output is then fed to a central binomial distribution (CBD) to generate small polynomial coefficients. The target for further experiments is the derivation of ρ and σ as seen in Algorithm 1 Line 1.

The ML-KEM PKE-Encrypt as shown in Algorithm 2 follows a similar structure where the coefficients for the secret polynomial y, e_1, e_2 are derived from a 32 byte random seed r that is generated and passed as an argument. The attack targets this random seed for achieving message recovery.

2.1.2. ML-DSA. ML-DSA is the NIST-standardised *Module-Lattice Digital Signature Algorithm* whose security rests on the Module Learning-with-Errors (MLWE) and Module Short Integer Solution (MSIS) problems. All polynomial operations are performed in the ring $R_q^{k \times \ell}$

Algorithm 2 ML-KEM PKE ENCRYPT (simplified)

Input: $\text{pk} \in \mathcal{B}^{384 \cdot k + 32}$,
Input: $m \in \mathcal{B}^{32}$,
Input: $r \in \mathcal{B}^{32}$
Output: ciphertext $c = (c_1 \| c_2)$
1: $(\hat{t}, \rho) \leftarrow Decode(pk)$
2: $\hat{A} \leftarrow \mathsf{ExpandA}(\rho)$
3: $(\mathbf{y}, \mathbf{e}_1, \mathbf{e}_2) \leftarrow \mathsf{DeriveSecrets}(r)$
4: $\hat{\mathbf{y}} \leftarrow \mathsf{NTT}(\mathbf{y})$
5: $\mathbf{w} \leftarrow \mathsf{NTT}^{-1}\left(\hat{A}^\top \circ \hat{\mathbf{y}}\right) + \mathbf{e}_1$
6: $\mathbf{u} \leftarrow \mathsf{Decompress}_{d_u}\left(\mathsf{ByteDecode}_{d_u}(m)\right)$
7: $\mathbf{v} \leftarrow \mathsf{NTT}^{-1}\left(\hat{t}^\top \circ \hat{\mathbf{y}}\right) + \mathbf{u} + \mathbf{e}_2$
8: $c_1 \leftarrow \mathsf{ByteEncode}_{d_u}\left(\mathsf{Compress}_{d_u}(\mathbf{w})\right)$
9: $c_2 \leftarrow \mathsf{ByteEncode}_{d_v}\left(\mathsf{Compress}_{d_v}(\mathbf{v})\right)$
10: **return** $c = (c_1 \| c_2)$

with $q = 2^{23} - 2^{13} + 1$. The value of the pair (k, ℓ) specifies the public-matrix dimension and determines the security category: ML-DSA-44 uses $(4, 4)$ for NIST level 2, ML-DSA-65 uses $(6, 5)$ for level 3, and ML-DSA-87 adopts $(8, 7)$ for level 5.

Algorithm 3 ML-DSA primitives (Simplified)

procedure KEYGEN()
$\xi \leftarrow \mathcal{B}^{32}$ ▷ 32-byte entropy
$(\rho, \rho', K) \leftarrow \mathsf{SHAKE256}(\xi)$
$\hat{A} \leftarrow \mathsf{ExpandA}(\rho)$ ▷ $\hat{A} \in R_q^{k \times \ell}$
$(\mathbf{s}_1, \mathbf{s}_2) \leftarrow \mathsf{CBD}(\rho')$
$\hat{t} \leftarrow \hat{A} \cdot \mathsf{NTT}(\mathbf{s}_1) + \mathsf{NTT}(\mathbf{s}_2)$
$(t_1, t_0) \leftarrow \mathsf{Power2Decompose}(\hat{t})$
return $pk = (\rho, t_1)$, $sk = (\mathbf{s}_1, \mathbf{s}_2, t_0, K, \rho)$
end procedure

procedure SIGN(sk, M) ▷ sk as returned by KEYGEN
unpack $sk \to (\mathbf{s}_1, \mathbf{s}_2, t_0, K, \rho)$
$\hat{A} \leftarrow \mathsf{ExpandA}(\rho)$
$\mu \leftarrow \mathsf{SHA3\_512}(\text{Tr}(pk) \parallel M)$
$rnd \leftarrow \mathcal{B}^{32}$
$\rho'' \leftarrow \mathsf{SHAKE256}(K \parallel rnd \parallel \mu)$ ▷ per-message seed
$\kappa \leftarrow 0$
repeat
$\mathbf{y} \leftarrow \mathsf{SampleMask}(\rho'', \kappa)$
$\mathbf{w} \leftarrow \hat{A} \cdot \mathsf{NTT}(\mathbf{y})$
$c \leftarrow \mathsf{HashToBall}(\mu, \mathbf{w})$
$\mathbf{z} \leftarrow \mathbf{y} + c\mathbf{s}_1$
$\mathbf{r} \leftarrow \mathbf{w} - c\mathbf{s}_2$
$\kappa \leftarrow \kappa + \ell$
until $\mathbf{z}, \mathbf{r}$ satisfy norm bounds
$h \leftarrow \mathsf{MakeHint}(\mathbf{r}, c, t_0)$
return $\sigma = (\mathbf{z}, c, h)$
end procedure

Algorithm 3 summarises the ML-DSA key-generation and signing procedures. The key-generation procedure expands a single 32-byte random data with SHAKE-256 to obtain three separate seeds: (i) a 32-byte seed ρ, used for sampling the coefficients of public matrix $\mathbf{A}$, (ii) a 64-byte seed ρ', from which the small secret vectors $\mathbf{s}_1$ and $\mathbf{s}_2$ are

sampled; and (iii) a 32-byte seed K that is used during the signing procedure.

The signing algorithm follows the *Fiat–Shamir-with-aborts* framework [16] where the signer repeatedly generates a candidate signature and discards it until a condition is satisfied. For every message $\mathbf{M}$ it first hashes the public-key tag tr together with the message $\mathbf{M}$ to generate μ. Then the per-message seed ρ'' is derived by concatenating the secret key component K, a fresh 32-byte random string rnd, and the hash μ. The core signing procedure involves sampling a vector $\mathbf{y}$ using ρ'', then multiplying it with the public matrix $\mathbf{A}$ to generate a commitment from which the challenge c is derived. The signature component $\mathbf{z} = \mathbf{y} + c \cdot \mathbf{s}_1$ where $\mathbf{s}_1$ taken from the secret key. If this $\mathbf{z}$ (and the vector $\mathbf{r}$) satifies the conditions, the signer computes a hint vector h and outputs the signature $\sigma = \big(\mathbf{z}, h, c\big)$. Otherwise it increments an internal counter κ and restarts the sampling procedure with the same seed ρ''. The main targets for further experiments are the generation of ρ''(Line 15) and sampling of y(Line 18).

2.2. Related Works

In the following, we briefly review prior fault attacks, applicable to ML-KEM and ML-DSA.

2.2.1. Attacks on KEMs. Fault attacks on ML-KEM can be broadly classified into two categories. The first category comprises attacks that are specific to individual cryptographic procedures, such as *key generation*, *encapsulation*, or *decapsulation*. These attacks exploit vulnerabilities unique to each operation's control flow or data handling. In contrast, the second category encompasses more generic attacks that target core components or primitives—such as polynomial arithmetic or random number generation—that are reused across multiple procedures. By inducing faults in these shared components, attackers can mount a variety of attacks across different phases of the cryptographic algorithm.

Fault attacks on key generation aims to generate a weakened LWE instance that directly exposes the secret key. Ravi *et al* [26] presented a multiple-fault attack that forces a nonce-reuse condition, turning the error vector e into s. The resulting instance takes the form $t = A \cdot s + s$, from which the secret vector s can be trivially recovered. Likewise, Espitau *et al.* [7] showed an attack on the Frodo key-exchange scheme which is similar to ML-KEM in which selected components of e are zeroed, revealing the secret key. These fault attacks are also applicable to the encapsulation procedure, that can result in message recovery attacks in Man-In-The-Middle (MITM) setting.

Fault attacks on *decapsulation* are most effective in a *static-key* setting, where the long-term secret key is reused. In an ephemeral-key setting, information leaked from a single execution vanishes with the key and is therefore useless to an attacker. All of the following works operate in that static-key context. Pessl and Prokop [22] demonstrate that flipping selected bits during message decoding and then observing the success/failure of the decapsulation routine leaks information about the secret key. Hermelink *et al.* [11] proposes an attack which involves injecting a *single bit flip* into one ciphertext coefficient. This attack can bypass countermeasures such as shuffled decoding and exposes the secret key. Delvaux [6] generalizes the attack to more relaxed fault models, at the cost of requiring a larger number of faulty executions for full key recovery. Finally, Xagawa *et al.* [34] shows that trivial *instruction-skip* faults in the decapsulation code can make the KEM return a shared secret even if the supplied ciphertext is invalid, thus bypassing the CCA protection.

Another category of fault attacks target specific algorithmic components. Ravi et al. [27] propose a novel fault attack on the Number Theoretic Transform (NTT) by zeroing out the twiddle constants by changing the pointer to point to memory containing zeroes, which significantly reduces the entropy of the NTT output. This reduction in entropy can facilitate key or message recovery in the Kyber KEM. Similarly, Wang et al. [30] present a practical attack on both ML-KEM and ML-DSA by targeting the Keccak operations using a loop-abort fault. Since Keccak is extensively utilized throughout the algorithms, this attack can compromise key generation, encapsulation, decapsulation.

2.2.2. Attacks on Signature Schemes. For signature schemes, the focus is on fault attacks that primarily target the signing and verification procedures. Attacks on key generation are often out of scope as key-generation is typically performed offline.

Fault attacks on the signing procedure typically target the critical operation $z = y + c \cdot s_1$ in ML-DSA. Bruinderink and Pessl [3] presented a differential fault attack on the deterministic variant of Dilithium, where the same nonce y is used for two signatures, one with the correct challenge c and one with a faulty c' and the difference between these signatures is used to recover the secret s_1. Similarly, Bindel et al. [2] proposed an instruction skip fault that omits the addition of y, thereby revealing coefficients of the product $s_1 \cdot c$, which can be used to recover s_1. Ravi et al. [24] reported a skip fault on a single coefficient of z, followed by a differential fault analysis (DFA) to recover s_1. Additionally, Bindel et al.[2] and Islam et al.[12] proposed random faults on the coefficients of the secret vector s_1. Finally, Espitau et al. [7] proposed a loop-abort fault targeting the sampling of the nonce y, zeroing out multiple coefficients and leading to secret recovery.

The verification routine offers far fewer fault-injection targets than the signing routine, so attackers typically focus on the *final check*. Bindel *et al.* [2] showed that forcing the challenge c to zero allows an invalid signature to pass verification. Ravi *et al.* [27] demonstrated this attack in practice by faulting the twiddle constant pointer.

Attacks such as the twiddle-constant fault of Ravi et al. [27] targeting the NTT operation or the fault attack proposed by Wang et al. [30] targeting the Keccak function can compromise *both* signing and verification procedures.

2.2.3. Motivation. When looking at how attackers try to break cryptographic systems using fault attacks, we often see that many techniques are tailored to specific algorithms. While some recent works have started focusing on common building blocks, like the NTT [27] and Keccak hash function [30], which are used across different algorithms, one crucial area has remained largely unexamined: the random seed. Modern post-quantum cryptographic designs, particularly those based on lattices and codes, heavily rely on a short, initial random seed. This seed is typically hashed to generate almost all the randomness needed for critical operations like key generation, data encapsulation, or digital signing. This design choice, while efficient, unfortunately turns the random seed into a single point of failure. If an attacker can corrupt this seed through a fault injection, the entire security of the cryptographic operation can be undermined. Therefore, it's critical to assess the random seed's susceptibility to fault-injection attacks and to devise robust countermeasures, as a successful breach could compromise a wide range of post-quantum algorithms.

3. Fault Vulnerabilities of Pointers

This section describes the experimental setup and the validated fault models.

3.1. Experiment Setup

Laser Fault Injection (LFI) was chosen as the main attack vector. The LFI setup uses $980nm$ diode laser. The laser supports a peak power of $2W$, a minimum spot size of 1.3μm and support pulse widths configurable from $1ns$ to 4μs. The device under test (DUT) chosen for the experiments is the STM32H753ZIT microcontroller with a ARM Cortex-M7 processor which implements the Armv7-M architecture. The setup also contains a software for setup synchronization, an oscilloscope for verifying the pulse characteristics and a relay for automated reset of the DUT.

3.2. Fault Characterization

An initial laser scan was performed throughout the surface of the STM32H753ZIT to find locations that are sensitive to laser faults. The initial scan was performed with 980nm laser source while a basic AES is running on the microcontroller. Majority of the faults were focused on the logic and memory regions. The area of the CPU highlighted with blue in Figure 1 is used for further experiments. This position was chosen since the faulty outputs were highly repeatable and this location allows direct targeting of specific assembly instructions.

Upon performing a detailed fault characterization on Memory and ALU operations, the following behaviors on ARM assembly instructions were observed.

1) Random bit flips in a $load$ instruction, modifying the value transferred to the destination register.

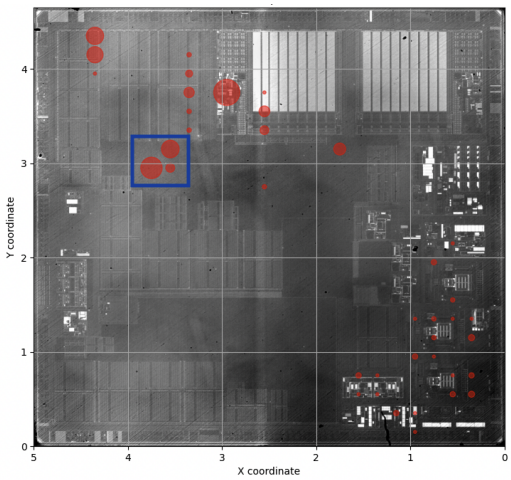


Figure 1: Fault sensitivity scan on the STM32H753ZI microcontroller for the 980nm laser

2) Instruction skips on $load$ or $store$ instructions, causing the destination register to retain its previous value.
3) Random bit flips in logical operations such as xor, resulting in corrupted data in the destination register.
4) Instruction skips on arithmetic operations such as add, again leaving the destination register unchanged.
5) Random faults in data-movement instructions like mov, modifying the value in the destination register.

3.3. Fault Model Definition

When an application uses an array or pointer, the CPU loads the base address into a register and then use this as reference for the subsequent memory access. The attack target is a single pointer register which modifies the content during the initialization, causing all later accesses to reference a *different* address while the program logic remains untouched.

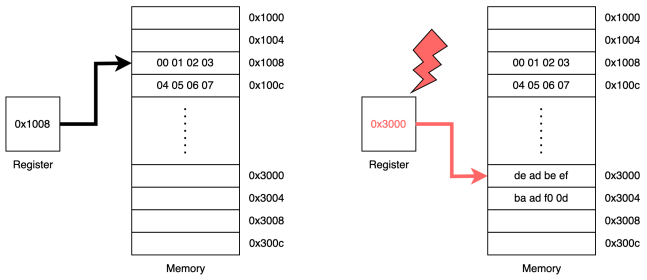
Let R_p denote a register that contains the address of a pointer and M be the memory map. A normal execution writes the intended base address α into R_p:

$$R_p = \alpha$$

The pointer-fault rewrites this value to $\alpha' \neq \alpha$:

$$R_p \xrightarrow{\text{fault}} R_p^{\star} = \alpha'$$

This faulty behavior is represented in the Figure 2. We discuss three main practical fault models to modify α to achieve various effects.



(a) Normal behavior (b) Faulty behavior

Figure 2: Memory accesses before and after a pointer–redirection fault.

3.3.1. Fault Model 1 (FM-1): Instruction-Skip on Addition. Let inp be a valid memory address and $c \in \mathbb{N}$ a compile-time constant. A correct assignment of the pointer out at an offset from inp would be as follows:

$$\texttt{out} \leftarrow \texttt{inp} + c$$

Fault Model 1 (FM-1) event occurs when the single machine instruction implementing the addition is skipped. The faulty pointer thus becomes:

$$\texttt{out}^\star = \texttt{inp}$$

So here the target pointer is initialized with the base address instead of initializing it at an offset.

Practical experiments. Consider the following code snippet:

```
1 int foo(uint8_t *array) {
2     /* Some prior code */
3     uint8_t *pointer = array + 0x10;
4     /* Remainder of the function */
5 }
```

The pointer assignment above is compiled to the following ARM assembly:

```
1  ldr  r3, [r7, #4]  // load address of array
2  adds r3, #16       // add offset
3  str  r3, [r7, #12] // store back to stack
```

First, the processor loads the address of the argument `array` from the stack into register `r3`. It then adds an offset of 16 bytes to advance the pointer. Finally, the updated address is written back to the stack for later use within the function. Here if we target the offset by skipping the $adds$ instruction, the $pointer$ gets assigned with the base address of the $array$.

This particular fault was achieved on the location marked in red in Figure 3. In a particular experiment, across all the faulty outputs, approximately 7.5% achieved this result. And for a particular set of parameters, a repeatability of 90–95 % was achieved.

3.3.2. Fault Model 2 (FM-2): Redirection to Zero-Filled Memory. Consider the pointer out to be loaded with an address $addr$

$$\texttt{out} \leftarrow \texttt{addr}$$

an *FM-2 fault* alters the effective address so that the loaded pointer becomes

$$\texttt{out}^\star = \texttt{addr}_Z,$$

where every byte in the target memory $\texttt{addr}_Z$ is a predictable value like `0x00`(or `0xff`). So any subsequent use of this pointer will cause all zeroes (or ones) to be used instead of the expected value.

Practical experiments. Consider the following C function call.

```
1 int foo(uint8_t *array1, uint8_t *array2 ) {
2   /* Some code */
3 }
4 int main(){
5   /* Some prior code */
6   foo(array1, array2);
7   /* Remainder of the function */
8 }
```

This function call is compiled to the following ARM assembly.

```
1 ldr r0, =array1 // store array1 to r0
2 ldr r1, =array2 // store array2 to r1
3 bl  foo         // call to foo()
```

Before the CPU calls a function, it loads the arguments into various registers. In this case, the addresses of $array1$ and $array2$ is loaded into $r0$ and $r1$ respectively. By targeting these load operations we can change the address loaded into the address leading to the argument being changed. Ravi *et al.* [27] achieved a similar fault model via electromagnetic fault injection for targeting the twiddle constant pointer.

This fault was obtained on the location marked in blue in Figure 3. Across several experiments, roughly 20% of the captured faults changed the register value in the required way. Using an optimal parameter set, the fault was 100% repeatable.

3.3.3. Fault Model 3: Redirection to Constant Memory. Consider the pointer out to be loaded with an address $addr$

$$\texttt{out} \leftarrow \texttt{addr}$$

An *FM-3 fault* diverts the address to a constant region $\texttt{addr}_C$:

$$\texttt{out}^\star = \texttt{addr}_C$$

where $out^\star$ points to a constant memory address. The attacker need not know about the data in the faulty pointer, but the idea is the data stays constant. This can be used to target the pointers that contains random data.

Practical experiments. Using the same physical hotspots as FM-1 and FM-2, we searched for faulty outputs in which the pointer referenced a memory location with constant data. For a suitable pulse width and timing offset, the experiment yielded constant memory faults with 90–100 % repeatability.

The overall repeatability of the three validated fault model is summarized in Table 2.

TABLE 2: Laser parameters and repeatability for each fault model

Fault model	Pulse width (ns)	Power (%)	Repeatability (%)
FM-1 ($out^{\star} \leftarrow addr_{\text{base}}$)	20–25	40–50	90–95
FM-2 ($out^{\star} \leftarrow addr_{Z}$)	25–47.5	30–55	100
FM-3 ($out^{\star} \leftarrow addr_{C}$)	20–22.5	40–55	90–100

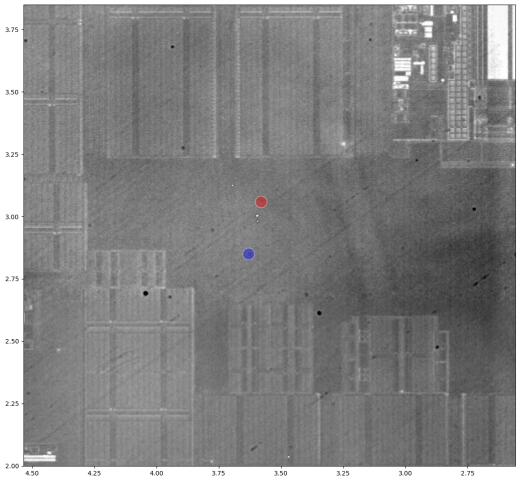


Figure 3: Locations of reliable Laser Fault Injection

4. Application to Post Quantum Cryptography

Prominent PQC algorithms fundamentally rely on expanding a small random seed to generate randomness. This makes them prime targets for the fault models we've demonstrated. By manipulating the memory pointer to this random seed, an attacker can modify its value, making subsequent algorithm operations (partially) deterministic and compromising security. Our experiments use optimized ML-KEM and ML-DSA implementations from the PQM4[13] library, a PQC benchmarking framework. This style of random seed handling is common across other major implementations like LibOQS[21], PQClean[14], and WolfSSL[32]; any significant differences influencing attacks will be noted. The implementation is compiled using `arm-none-eabi-gcc` using `-mcpu=cortex-m7 -O0 -mfpu=fpv5-d16 -mfloat-abi=hard -mthumb` options.

4.1. Threat model

Our threat model assumes an attacker has direct physical access to the Device Under Test (DUT), allowing them to decapsulate its package and expose the processor's silicon without further invasive modification. This is a standard assumption for LFI. Within this setup, the attacker can precisely observe the DUT's behavior and monitor its external communication channels. They possess a good understanding of the timing for each cryptographic operation, although they do not need precise triggers for the exact operations. The attacker is capable of injecting a single laser pulse per cryptographic operation. While they can profile an identical device beforehand to determine the optimal fault injection parameters, during the actual attack, they are limited to information available on the communication channel and cannot probe internal registers or memory directly.

4.2. Attacks on ML-KEM

ML-KEM begins key generation by expanding a single 32-byte entropy source into two domain-separated seeds: the public seed ρ, which defines the public matrix, and the private seed σ, which are used to derive the sampling of secret and error vectors. Since subsequent steps depend on these two seeds, any of the three fault models described in the previous section can be applied to compromise the scheme. Consider the following code snippet corresponding to seed initialization in the *ML-KEM key-generation* function:

```
1  void indcpa_keypair(unsigned char *pk,
2                      unsigned char *sk,
3                      const unsigned char *coins){
4
5      unsigned char buf[2 * KYBER_SYMBYTES];
6      unsigned char *publicseed = buf;
7      unsigned char *noiseseed = buf + KYBER_SYMBYTES;
8
9      memcpy(buf, coins, KYBER_SYMBYTES);
10     buf[KYBER_SYMBYTES] = KYBER_K;
11     hash_g(buf, buf, KYBER_SYMBYTES + 1);
12     // ...
13     // Remainder of function
14 }
```

Listing 1: ML-KEM PKE key generation seed initialization

Here, the 32-byte random seed (`coins`) is passed to the keygen function. This seed is expanded to create the public and private seed using the G function (SHA3-512). The hash output is stored in the `buf` array. The data in `buf` is divided in half by initializing the pointers `publicseed` to the start and `noiseseed` to the midpoint of the `buf` array. Next, we describe how the `noiseseed` initialization can be targeted using the three defined fault models: FM-1, FM-2 and FM-3.

4.2.1. Attack 1: FM-1 on `noiseseed` initialization.

The attack scenario involves targeting the addition operation using FM-1 while the `noiseseed` is being initialized as seen in Line 7. By skipping the addition operation during this initialization, we effectively point the `noiseseed` to the start of `buf`. This will force the computation of secret and error vectors with the `publicseed`. As `publicseed` is known, the attack results in secret key recovery.

Consider Line 7 in Listing 1. The line is compiled to the following assembly instructions.

```
1  movw    r3, #5136
2  add     r3, r7
3  adds    r3, #32
4  movw    r2, #5200
5  add     r2, r7
6  str     r3, [r2, #0]
```

The function first allocates the local array `buf`. Then its base address is loaded into registers via `mov` instructions to initialize the relevant pointers. The fault attack targets the subsequent `adds` instruction which initializes the `noiseseed` pointer at an offset. By skipping this single instruction, the address stored in `r3` is never incremented, making `noiseseed` and `publicseed` points to the same memory location. Here, each successful fault leads to key recovery.

Practical Experiments. For a set of optimal parameters, 466 of 500 key-generation attempts produced a successful fault that lead to key recovery, yielding a success rate of **93.2%**. The optimal parameters are discussed in Table 3

4.2.2. Attack 2: FM-2 on `coins` memcpy. This attack directly targets the loading of the address into the `noiseseed` pointer using FM-2. A fault at this point leads to a change in the address stored in the pointer. By redirecting the pointer to a memory region filled with predictable values—such as all zeros—an attacker can effortlessly recover the key. Similar results can be achieved by faulting the `coins` pointer during the key generation function call.

We target the `memcpy` function as shown in line 9 of Listing 1. The specific point of interest is the moment just before the function call, when the function arguments are loaded into the corresponding registers. The assembly instructions for this code are as follows:

```
1 movw    r0, #5136
2 add     r0, r7
3 movs    r2, #32
4 ldr     r1, [r3, #0]
5 bl      0x800b722 <memcpy>
```

The function initially loads the pointer `coins` from the stack into register `r1` using `ldr` instruction. The fault corrupts this specific instruction such that, instead of the correct address, an address referring to a memory region pre-initialized with predictable data (e.g., `0x00` or `0xFF`) is loaded. A successful fault therefore causes the algorithm to generate a *known* key.

A message recovery attack can be mounted by corrupting the random seed passed to `indcpa_enc`. The `coins` variable is copied to a temporary buffer `buf`, then hashed to derive the per-message seed. By applying the same pointer-fault technique used in the key generation attack, specifically targeting the `memcpy` call in Listing 2 Line 8, an attacker can fix the seed value and control the randomness of encapsulation. Ravi et al. [27] demonstrated how such faults can be exploited in a man-in-the-middle scenario to recover the message while preserving the correctness of the exchange.

Practical Experiments. For a set of optimal parameters, 500 of 500 key generation attempts produced a successful fault that lead to key recovery, yielding a success rate of **100%**.

```
1  int crypto_kem_enc_derand(uint8_t *ct,
2                            uint8_t *ss,
3                            const uint8_t *pk,
4                            const uint8_t *coins) {
5      uint8_t buf[2 * KYBER_SYMBYTES];
6      uint8_t kr[2 * KYBER_SYMBYTES];
7
8      memcpy(buf, coins, KYBER_SYMBYTES);
9
10     hash_h(buf + KYBER_SYMBYTES, pk,
           KYBER_PUBLICKEYBYTES);
11     hash_g(kr, buf, 2 * KYBER_SYMBYTES);
12
13     indcpa_enc(ct, buf, pk, kr + KYBER_SYMBYTES);
14
15     memcpy(ss, kr, KYBER_SYMBYTES);
16     return 0;
17 }
```

Listing 2: ML-KEM key encapsulation

4.2.3. Attack 3: FM-3 on `sigma` initialization. This attack is similar to the previously described scenario but assumes the attacker cannot force `sigma` to a predictable values. Instead, the attacker injects a fault that causes the seed pointer to read from a fixed memory location whose contents are constant across faulty executions. Repeating the fault over two Key Generation yields the *same* secret vectors s, e while producing independent public matrices A_1 and A_2, giving $t_1 = A_1 \cdot s + e$ and $t_2 = A_2 \cdot s + e$. Subtracting eliminates the error resulting in $t_1 - t_2 = (A_1 - A_2) \cdot s$, from which the secret key can be retrieved.

Practical Experiments:. Using a fixed set of experimentally optimized parameters, 467 out of 500 key generation attempts resulted in a fixed key, yielding a **93.4%** success rate. The optimal parameters are discussed in Table 3 and the corresponding fault injection location is indicated in Figure 3.

TABLE 3: Laser parameters and success rate for each attack

Attack	Pulse width (ns)	Power (%)	Offset (ns)	Repeat. (%)
Attack 1	25	45	769	93.2
Attack 2	25	55	991	100
Attack 3	22.5	50	718	100

Effect on existing countermeasures. The fault alters only the initialization of the random seed, leaving all subsequent steps of the algorithm unchanged. Most proposed countermeasures [23] protect specific functional blocks, such as the NTT or decoding routine, but do not safeguard the seed pointer itself. As a result, our attack bypasses these defenses.

4.3. Attacks on ML-DSA

Having established how pointer faults on the seed undermine ML-KEM, we now turn to the lattice-based signature

scheme ML-DSA. Signatures are an attractive target, as the private signing key is long-lived and the signing operation is executed in the field, away from the authority that generated the key. In ML-DSA, each signature begins by generating a per-message random seed. Corrupting this value using the previously introduced fault models can enable recovery of the signing key.

Consider the following code snippet corresponding to sign function in ML-DSA:

```
1  int crypto_sign_signature_ctx(uint8_t *sig,
2                                size_t *siglen,
3                                const uint8_t *m,
4                                size_t mlen,
5                                const uint8_t *ctx,
6                                size_t ctxlen,
7                                const uint8_t *sk) {
8    //
9    /* Initialize local variables*/
10   //
11   /* mu = CRH(tr, 0, ctxlen, ctx, msg) */
12   //
13   randombytes(rnd, RNDBYTES);
14   shake256(rhoprime, CRHBYTES, key, SEEDBYTES +
         RNDBYTES + CRHBYTES);
15   //
16   /* Expand matrix and transform vectors */
17   //
18 rej:
19   /* Sample intermediate vector y */
20   polyvecl_uniform_gamma1(&y, rhoprime, nonce++);
21   // ...
22   // Remainder of function
23 }
```

Listing 3: ML-DSA Sign Simplified

There are two main attacks that target the `rhoprime` seed variable.

4.3.1. Attack 1: FM-2 on sampling of rhoprime or y. This attacks involves targeting the generation of the intermediate vector y using FM-2. This can be done by targeting $rhoprime$ initialization in Listing 3 Line 14 or the sampling of y in line 20. By performing this fault attack we can fix the intermediate vector y in the signing process. By fixing y to a known value, the attacker can retrieve the partial secret key s_1.

The function call in Listing 3 Line 14 is compiled to the following assembly instructions.

```
1 movs    r3, #128
2 ldr.w   r2, [r7, #3828]
3 movs    r1, #64
4 ldr.w   r0, [r7, #3816]
5 bl      0x8004ccc <shake256>
```

This function loads the input to the shake function into the register `r2`. By applying Fault Model FM-2 to this `ldr` instruction, the attacker redirects the pointer so that the output of SHAKE256, stored at `rhoprime`, becomes a predictable byte string. As `rhoprime` is subsequently used to sample the nonce vector y, the attacker now knows y. Given y, together with the publicly known signature components z and c, partial secret key s_1 can be recovered.

Some implementations like LibOQS, WolfSSL initialize $rhoprime$ differently where the input to `shake256` is not a single pointer, as seen in Listing 4

```
1 /* Compute rhoprime = CRH(key, rnd, mu) */
2 shake256_inc_ctx_reset(&state);
3 shake256_inc_absorb(&state, key, SEEDBYTES);
4 shake256_inc_absorb(&state, rnd, RNDBYTES);
5 shake256_inc_absorb(&state, mu, CRHBYTES);
6 shake256_inc_finalize(&state);
7 shake256_inc_squeeze(rhoprime, CRHBYTES, &state);
```

Listing 4: LibOQS rhoprime generation

This can make it difficult to target the `rhoprime` initialization with a single fault. In this case, one can target the sampling of y as seen in Listing 3 Line 20. This attack is applicable when the signing is completed within the first iteration. We tested 1 million signatures and found that around 23% of signing is completed in the first iteration, as shown in Figure 4.

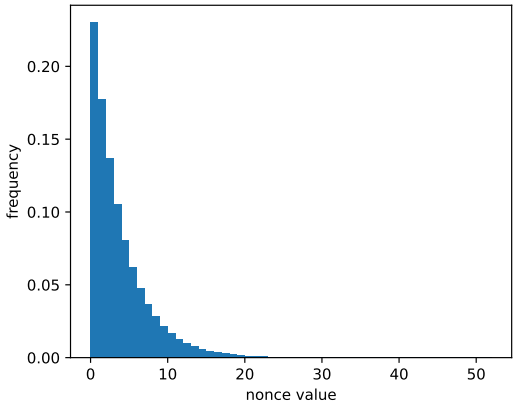


Figure 4: ML-DSA nonce distribution across 1,000,000 signatures

4.3.2. Attack 2: Differential attack using FM-3. A similar exploit can be obtained with FM-3.

This attack aims at generating a fixed value for the intermediate vector y using FM-3. Recall that with FM-3, the adversary is able to force the memory pointer to a constant but unknown value. In this case, the adversary would need to perform a differential analysis over two faulty signatures. For two signatures, the fault is injected under model FM-3 to force the value of y to y'. The two signatures need not be consecutive but must complete signing in first iteration. Thus, we have $z_1 \leftarrow y' + c_1 \cdot s_1$ and $z_2 \leftarrow y' + c_2 \cdot s_1$. As (z_1, c_1) and (z_2, c_2) are public parameters, a simple differential analysis allows the adversary to recover partial secret key s_1. It was shown in [25] that recovering s_1 alone allows an adversary to forge signatures.

Experimental Validation. Our experiments confirmed that when targeting the sampling of y (Listing 3, line 20), an average of *8 to 9* signing attempts is required to obtain the two faulty signatures needed for key-recovery.

Remark. FM-2 and FM-3 could potentially target ML-DSA key generation; however, this is unlikely in practice since

key generation is usually performed offline. FM-1 cannot be applied to ML-DSA key generation because of the difference in size between the public seed (32 bytes) and the secret seed (64 bytes), limiting the attacker to retrieving only part of the secret seed.

Effect on existing countermeasures. Since the attack targets the sampling of the vector y and not any core computation of the signing process, the fault will still produce a valid signature. Therefore, generic countermeasures, such as verifying the signature after generation, will not detect this fault.

4.4. Effects of Compiler Optimizations

The locations susceptible to our fault models depend on both the implementation of the algorithm, and the way the compiler translates that code into machine instructions, there is a need to determine whether the same weakness persists at higher optimizations.

Consider the following assembly listing, which shows the compiled code for the pointer initialization in ML-KEM at *O3*:

```
1  ldr     r0, [r2, #0]
2  mov     r9, r1
3  mov     r12, sp
4  ldr     r1, [r2, #4]
5  mov.w   r10, #2
6  ldr     r6, [r3, #0]
7  add.w   r8, sp, #32
8  ldr.w   r3, [lr, #12]
9  mov     r7, sp
10 ldr     r2, [r2, #8]
11 stmia.w r12!, {r0, r1, r2, r3}
12 ldr.w   r0, [lr, #16]
13 ldr.w   r1, [lr, #20]
14 ldr.w   r2, [lr, #24]
15 ldr.w   r3, [lr, #28]
16 stmia.w r12!, {r0, r1, r2, r3}
17 movs    r2, #33 @ 0x21
18 mov     r1, sp
19 mov     r0, sp
20 strb.w  r10, [sp, #32]
21 bl      0x8001688 <sha3_512>
```

With optimisation level `-O3` the compiler uses the `add.w` instruction (line 7) to initialize the pointer `noiseseed` as `r8` $\leftarrow$ `sp` $+ 32$. Applying FM-1 here is no longer a simple *instruction skip*: skipping the `add.w` would remove the assignment entirely, not leave `r8` unchanged. Instead, the attacker must target the internal addition so that the destination register receives `sp` (contains the base address of `buf`) rather than `sp` $+ 32$, to load `noiseseed` with the same address as `publicseed`. Although the fault becomes more complex and requires finer control at higher optimization levels, the vulnerability exploited by FM-1 remains present.

At higher optimization levels, the `memcpy` function is replaced by a sequence of `ldr` and `stmia` instructions. Consequently, the most convenient fault injection point for FM-2 and FM-3 shifts to the argument initialization for the `sha3_512` call. Here, the base address of `buf` is moved into registers `r0` and `r1` using simple `mov` instructions, instead of the `ldr` targeted at `-O0`. Although our experiments yielded better results when faulting `ldr`, the `mov` at line 9 remains a viable and vulnerable target. In fact, FM-3 on ML-KEM successfully exploits this `mov` instruction at `-O0`, as shown in the listing in Section 4.2.1. The same argument initialization pattern appears in the ML-DSA signing function calls, making the attack transferable to ML-DSA.

5. Countermeasures

The experiments clearly show that the random seed is a prime target for fault-injection attacks; corrupting it allows practical attacks against several post-quantum cryptographic schemes. Although software protections like Pointer Authentication[15] and Address Space Layout Randomization (ASLR)[29] can make these attacks harder to repeat, they depend on an operating system kernel and a memory management unit. These components are usually missing in most low-end microcontrollers. This highlights a clear need for lightweight, software-level protections specifically designed to secure the seed pointer on these resource-constrained devices. The rest of this section will detail dedicated software countermeasures that can prevent the types of attacks we've identified.

5.1. FM-1 on ML-KEM

One set of countermeasures can be deployed to significantly reduce the probability of noiseseed and publicseed holding identical values, which is critical for the success of FM-1. These countermeasure may explore:

1) **Sanity check on `noiseseed`:** Given that the attack's success relies on `publicseed` and `noiseseed` containing identical data, a straightforward comparison can be integrated. This check would verify the distinctness of the two buffers before the private key sampling process commences, thereby preventing the malicious equivalence from being exploited.
2) **Storing the seeds in separate buffers:** The vulnerability stems from the proximity of the two pointers within a single allocated buffer. To mitigate this, instead of generating both `publicseed` and `noiseseed` concurrently within one buffer (e.g., via SHA3-512), we propose generating them into entirely distinct memory locations. This could be achieved by using separate cryptographic primitives, such as SHAKE-256, for each seed's generation and storage, thus eliminating their adjacency in memory.

5.2. FM-2 on ML-KEM and ML-DSA

These countermeasures aim to significantly reduce the likelihood of the random seed being populated with a predictable value, thereby preventing FM-2.

1) **Entropy check on the random seed:** When a fault injection manipulates the random seed to a predictable value (e.g., an all-zeros), the resulting data will exhibit a substantially lower entropy than a genuinely random seed. Implementing a lightweight entropy check can effectively detect such maliciously modified seeds, allowing the system to reject them and prevent further exploitation.
2) **Seed Blacklisting:** As the adversary can redirect memory pointers to only a limited set of known memory locations, the range of predictable seed values resulting from such redirection is finite and often small. By maintaining a blacklist of these identified 'bad' or predictable seed values, the system can detect and reject them if they are generated, thereby mitigating this specific attack vector.

5.3. FM-3 on ML-DSA

These countermeasures are designed to increase the difficulty for an adversary to force a random seed to a constant, predictable value through fault injection.

1) **Timing Jitter and Horizontal Noise:** Forcing a seed pointer to a specific, constant memory address necessitates a fault injection that is precisely timed. Introducing a minor, data-independent delay, either immediately preceding or following the computation of ρ'', or during the sampling of y, can effectively disrupt this precise timing. Such 'jitter' misaligns the attacker's required fault window, significantly reducing the fault repeatability and thus the success rate.
2) **Pointer Redundancy:** Each seed pointer can be loaded into a secondary, dummy register. Prior to the actual use of the seed pointer, its value is compared against the value stored in this dummy register. Any detected mismatch indicates a successful fault injection, allowing the cryptographic routine to safely abort execution and prevent the exploitation of the corrupted pointer.

While these countermeasures do not eliminate the attack surface entirely, they enhance the security of post-quantum cryptographic implementations. They offer lightweight, software-only mitigations that are particularly valuable for resource-constrained environments like microcontrollers, where more complex hardware-based defenses are often infeasible. By introducing an additional layer of defense against precise fault-injection attacks, these techniques can significantly reduce the practical success rate by several orders of magnitude, making it substantially more difficult and time-consuming for an adversary to compromise the integrity of the random seed and, consequently, the cryptographic operations.

6. Conclusions

This paper demonstrates that fault attacks on the seed pointer enable practical exploits against ML-KEM and ML-DSA implementations. We introduced three fault models that achieve key- or message-recovery attacks on ML-KEM and signature-forgery attacks on ML-DSA, with a 90–100% success rate on optimized ARM Cortex-M7 implementations. Since most post-quantum schemes generate its required randomness by expanding a short seed with hash functions, the same attack methodology can be extended to other candidates, such as *BIKE*[1] and *HQC*[17], which will be explored in future work.

References

[1] Nicolas Aragon, Paulo L. Barreto, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Santosh Ghosh, Shay Gueron, Tim Güneysu, Carlos Aguilar Melchor, Rafael Misoczki, Edoardo Persichetti, Jan Richter-Brockmann, Nicolas Sendrier, Jean-Pierre Tillich, Valentin Vasseur, and Gilles Zémor. BIKE: Bit flipping key encapsulation – algorithm specification. Submission to the NIST Post-Quantum Cryptography Standardization Project (Round 4), October 2024.

[2] Nina Bindel, Johannes Buchmann, and Juliane Krämer. Lattice-based signature schemes and their sensitivity to fault attacks. In *2016 Workshop on Fault Diagnosis and Tolerance in Cryptography, FDTC 2016, Santa Barbara, CA, USA, August 16, 2016*, pages 63–77. IEEE Computer Society, 2016.

[3] Leon Groot Bruinderink and Peter Pessl. Differential fault attacks on deterministic lattice signatures. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(3):21–43, 2018.

[4] Lily Chen, D Moody, and Y Liu. Nist post-quantum cryptography standardization. *Transition*, 800(131A):164, 2017.

[5] Cloudflare. Post-quantum cryptography 2024. https://blog.cloudflare.com/pq-2024/, April 2024. Accessed: 2025-06-09.

[6] Jeroen Delvaux. Roulette: A diverse family of feasible fault attacks on masked kyber. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(4):637–660, 2022.

[7] Thomas Espitau, Pierre-Alain Fouque, Benoît Gé rard, and Mehdi Tibouchi. Loop-abort faults on lattice-based signature schemes and key exchange protocols. *IEEE Trans. Computers*, 67(11):1535–1549, 2018.

[8] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. FN–DSA (falcon): Fast-fourier lattice-based compact signatures over ntru — algorithm specification. Submission to the NIST Post-Quantum Cryptography Standardization Project, October 2024. Candidate standard for FIPS 206.

[9] Craig Gidney. How to factor 2048 bit rsa integers with less than a million noisy qubits. *arXiv preprint arXiv:2505.15917*, 2025.

[10] Google Security Blog. A new path for kyber on the web. https://security.googleblog.com/2024/09/a-new-path-for-kyber-on-web.html, September 2024. Accessed: 2025-06-09.

[11] Julius Hermelink, Peter Pessl, and Thomas Pöppelmann. Fault-enabled chosen-ciphertext attacks on kyber. In Avishek Adhikari, Ralf Küsters, and Bart Preneel, editors, *Progress in Cryptology - INDOCRYPT 2021 - 22nd International Conference on Cryptology in India, Jaipur, India, December 12-15, 2021, Proceedings*, volume 13143 of *Lecture Notes in Computer Science*, pages 311–334. Springer, 2021.

[12] Saad Islam, Koksal Mus, Richa Singh, Patrick Schaumont, and Berk Sunar. Signature correction attack on dilithium signature scheme. In *7th IEEE European Symposium on Security and Privacy, EuroS&P 2022, Genoa, Italy, June 6-10, 2022*, pages 647–663. IEEE, 2022.

[13] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Post-quantum cryptography library for the arm cortex-m4. GitHub repository, October 2024.

[14] Matthias J. Kannwischer, Peter Schwabe, Douglas Stebila, Thom Wiggers, et al. PQClean: Clean, portable, tested implementations of post-quantum cryptography. GitHub repository, June 2025.

[15] Hans Liljestrand, Thomas Nyman, Kui Wang, Carlos Chinea Perez, Jan-Erik Ekberg, and N. Asokan. PAC it up: Towards pointer integrity using ARM pointer authentication. In Nadia Heninger and Patrick Traynor, editors, *28th USENIX Security Symposium, USENIX Security 2019, Santa Clara, CA, USA, August 14-16, 2019*, pages 177–194. USENIX Association, 2019.

[16] Vadim Lyubashevsky. Fiat-shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *Advances in Cryptology - ASIACRYPT 2009, 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6-10, 2009. Proceedings*, volume 5912 of *Lecture Notes in Computer Science*, pages 598–616. Springer, 2009.

[17] Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jurjen Bos, Jean-Christophe Deneuville, Arnaud Dion, Philippe Gaborit, Jérôme Lacan, Edoardo Persichetti, Jean-Marc Robert, Pascal Vé ron, and Gilles Zémor. HQC: Hamming quasi-cyclic public-key encryption and key-encapsulation mechanism – algorithm specification. Submission to the NIST Post-Quantum Cryptography Standardization Project, February 2024.

[18] National Institute of Standards and Technology. Module–lattice-based digital signature algorithm (ml-dsa). FIPS Publication 204, U.S. Department of Commerce, National Institute of Standards and Technology, aug 2024. Final standard, August 2024.

[19] National Institute of Standards and Technology. Module–lattice-based key-encapsulation mechanism (ml-kem). FIPS Publication 203, U.S. Department of Commerce, National Institute of Standards and Technology, aug 2024. Final standard, August 2024.

[20] National Institute of Standards and Technology. Stateless hash-based digital signature standard. Federal Information Processing Standards Publication FIPS 205, U.S. Department of Commerce, Washington, D.C., August 2024. Final version, August 13 2024.

[21] Open Quantum Safe Project. liboqs: C library for quantum-safe cryptographic algorithms. GitHub repository, April 2025.

[22] Peter Pessl and Lukas Prokop. Fault attacks on cca-secure lattice kems. *IACR Cryptol. ePrint Arch.*, page 64, 2021.

[23] Prasanna Ravi, Anupam Chattopadhyay, Jan-Pieter D'Anvers, and Anubhab Baksi. Side-channel and fault-injection attacks over lattice-based post-quantum schemes (kyber, dilithium): Survey and new results. *ACM Trans. Embed. Comput. Syst.*, 23(2):35:1–35:54, 2024.

[24] Prasanna Ravi, Mahabir Prasad Jhanwar, James Howe, Anupam Chattopadhyay, and Shivam Bhasin. Exploiting determinism in lattice-based signatures: Practical fault attacks on pqm4 implementations of NIST candidates. In Steven D. Galbraith, Giovanni Russello, Willy Susilo, Dieter Gollmann, Engin Kirda, and Zhenkai Liang, editors, *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security, AsiaCCS 2019, Auckland, New Zealand, July 09-12, 2019*, pages 427–440. ACM, 2019.

[25] Prasanna Ravi, Mahabir Prasad Jhanwar, James Howe, Anupam Chattopadhyay, and Shivam Bhasin. Exploiting determinism in lattice-based signatures: practical fault attacks on pqm4 implementations of nist candidates. In *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security*, pages 427–440, 2019.

[26] Prasanna Ravi, Debapriya Basu Roy, Shivam Bhasin, Anupam Chattopadhyay, and Debdeep Mukhopadhyay. Number "not used" once - practical fault attack on pqm4 implementations of NIST candidates. In Ilia Polian and Marc Stöttinger, editors, *Constructive Side-Channel Analysis and Secure Design - 10th International Workshop, COSADE 2019, Darmstadt, Germany, April 3-5, 2019, Proceedings*, volume 11421 of *Lecture Notes in Computer Science*, pages 232–250. Springer, 2019.

[27] Prasanna Ravi, Bolin Yang, Shivam Bhasin, Fan Zhang, and Anupam Chattopadhyay. Fiddling the twiddle constants - fault injection analysis of the number theoretic transform. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(2):447–481, 2023.

[28] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6):34:1–34:40, 2009.

[29] Kevin Z. Snow, Fabian Monrose, Lucas Davi, Alexandra Dmitrienko, Christopher Liebchen, and Ahmad-Reza Sadeghi. Just-in-time code reuse: On the effectiveness of fine-grained address space layout randomization. In *2013 IEEE Symposium on Security and Privacy, SP 2013, Berkeley, CA, USA, May 19-22, 2013*, pages 574–588. IEEE Computer Society, 2013.

[30] Yuxuan Wang, Jintong Yu, Shipei Qu, Xiaolin Zhang, Xiaowei Li, Chi Zhang, and Dawu Gu. Beware of keccak: Practical fault attacks on SHA-3 to compromise kyber and dilithium on ARM cortex-m devices. *IACR Cryptol. ePrint Arch.*, page 1522, 2024.

[31] wolfSSL. wolfcrypt with post-quantum cryptography. https://www.wolfssl.com/products/wolfcrypt-post-quantum/, 2024. Accessed: 2025-06-09.

[32] wolfSSL Inc. wolfSSL: Embedded tls library with post-quantum cryptography support. Software release announcement, March 2024. Adds Kyber, ML-DSA and other PQC ciphers.

[33] World Economic Forum. Y2q: What is it and why does it matter? https://www.weforum.org/stories/2023/10/y2q-cybersecurity-cyberattack-quantum-computing/, October 2023. Accessed: 2025-06-09.

[34] Keita Xagawa, Akira Ito, Rei Ueno, Junko Takahashi, and Naofumi Homma. Fault-injection attacks against nist's post-quantum cryptography round 3 KEM candidates. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part II*, volume 13091 of *Lecture Notes in Computer Science*, pages 33–61. Springer, 2021.

[35] Zoom Video Communications. A guide to post-quantum end-to-end encryption. https://www.zoom.com/en/blog/guide-to-post-quantum-end-to-end-encryption/, 2024. Accessed: 2025-06-09.